Experiment – 8

Name: SIMRAN CHOUHAN

UID: 24bai70124

Section: 24AIT_KRG G1

A. Experimental Procedure

Step 1: Implementation

Binary Search,

Subset Sum (Verification and Decision versions),

Traveling Salesman Problem (Brute Force).

The implementation should ensure correctness, proper handling of edge cases, and readability.

Step 1: Implementation

The implementation phase focused on developing three distinct algorithms to observe their behavior across different complexity classes. Each was coded to ensure high readability and robust handling of edge cases.

Binary Search (O(log n)):

- Approach: Implemented using an iterative divide-and-conquer strategy on a pre-sorted array.
- Logic: The algorithm repeatedly identifies the middle element and discards the half of the array that cannot contain the target.
- Edge Cases: Handled scenarios involving empty arrays, single-element arrays, and targets that were either smaller than the minimum or larger than the maximum element.

Subset Sum (Verification and Decision):

- Decision Version (O(2^n)): Developed using recursion with memoization to determine if any combination of numbers in a set adds up to a target value K.

- Verification Version (O(n)): Implemented a simple linear-time function that takes a “candidate subset” and validates if its sum equals K.
- Edge Cases: Addressed inputs where the target K is zero, cases with negative integers (if applicable), and sets where the sum of all elements is still less than the target.

Traveling Salesman Problem (Brute Force - O(n!)):

- Approach: Utilized a permutation-based strategy to calculate the total distance of every possible Hamiltonian cycle in a graph.
- Logic: The implementation iterates through all (n-1)! possible paths, keeping track of the one with the minimum total weight.
- Edge Cases: Handled small graphs (2–3 nodes), disconnected graphs, and weighted edges to ensure the logic remained sound regardless of graph density.

Step 2: Measure Metrics

Record key performance metrics for each execution, including execution time, approximate number of operations, and feasibility status (Completed or Timeout). Maintain a structured log for further analysis.

To analyze the “Factorial Wall” and “Exponential Growth,” performance metrics were logged during execution.

- **Execution Time:** Recorded using high-precision timers (milliseconds) to capture the difference between sub-millisecond search results and multi-second brute-force calculations.
- **Approximate Number of Operations:** Calculated based on the Big O theoretical growth (n! for TSP, 2^n for Subset Sum, and log n for Binary Search) to correlate theoretical complexity with actual CPU work.
- **Feasibility Status: Completed:** Assigned to executions where the algorithm finished within a 30-second window.
- **Timeout:** Assigned to instances where the combinatorial explosion made it impossible for the computer to finish (typically seen in TSP with n > 15).

B. Analysis Tasks (Critical Thinking)

Q1. Why does Binary Search consistently demonstrate efficient performance across large input sizes?

Ans. Binary Search stays efficient even for very large datasets because its time complexity grows logarithmically, O(log n). Instead of checking each element one by one like Linear Search, it keeps cutting the search space into half after every comparison. For example, increasing the input size from thousands to millions only adds a few extra steps. This controlled growth is what makes it practical and scalable.

Q2. Explain why Subset Sum is computationally difficult to solve but relatively easy to verify.

Ans. Subset Sum is hard to solve because it may require checking every possible subset, which grows exponentially (2^n). As n increases, the number of combinations becomes extremely large, making brute force impractical. However, verifying a solution is easy. If a subset is already given, we just calculate its sum and compare it with the target, which only takes linear time O(n). So finding a solution is difficult, but checking one is straightforward.

Q3. Identify the input size at which the Traveling Salesman Problem becomes infeasible and justify the reason.

Ans. In brute-force TSP, the problem becomes impractical somewhere around 12 to 15 cities. This is because the number of possible routes increases factorially (n!). Even a small increase in input size causes a massive jump in computations. For example, with around 13 cities, the number of paths already reaches billions. Beyond this point, the execution time becomes too large for normal systems, making the approach infeasible.

Q4. Differentiate between solving a problem and verifying a given solution with appropriate examples.

Ans. Solving a problem means actively searching for a valid solution among many possibilities without knowing the answer beforehand.
Example: Finding a subset whose sum equals a target value.
Verifying a solution means checking whether a given answer satisfies the conditions of the problem.
Example: Checking if a provided subset actually adds up to the target.

Example: Find subset with sum = target

Verifying a Solution: This involves checking if a provided answer satisfies all the problem's constraints.

Example: Check if given subset sums to target

Q5. Discuss why NP-Complete problems are considered the most challenging class within NP.

Ans. NP-Complete problems are considered the toughest in NP because no efficient (polynomial-time) solution is known for them. They usually involve exploring a huge number of possibilities, which grows exponentially with input size. Another important point is that if any one NP-Complete problem is solved efficiently, it would mean all NP problems can also be solved efficiently. Common examples include Subset Sum and Traveling Salesman Problem.

No known polynomial-time solution exists

If one NP-Complete problem is solved efficiently, all NP problems can be solved efficiently

They require checking an exponential number of possibilities

Examples: Subset Sum (Decision), Traveling Salesman Problem